
Verification-Gated Hierarchical Execution: A Demo for Long-Horizon Agents

Anonymous Author(s)

Affiliation

Address

email

1 Submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026), workshop: Who Verifies the
2 Agents? Toward Reliable Agent Development. Do not distribute.

Abstract

3
4 Long-horizon agents can fail after many locally plausible steps: a completion is unsupported,
5 an intermediate artifact becomes stale, or a repair silently invalidates downstream work. Post-hoc
6 scoring identifies some failures but leaves execution control unresolved: what was checked, what
7 evidence supports it, and where should execution resume? We present a runnable, package-
8 resident agent harness in which persistent intent is refined through *Goal* $\rightarrow$ *Plan* $\rightarrow$ *Phase* $\rightarrow$
9 *durable intent contract (DIC)* $\rightarrow$ *disposable execution prompt stack (EPS)* $\rightarrow$ *Prompt/Run* $\rightarrow$
10 *Action*, actions bind to evidence, and verification is a control signal with four outcomes: accept,
11 repair, replan, or escalate. The implementation records recoverable lineage and lets a failed
12 phase instantiate a new bounded execution stack while preserving its durable contract. In a
13 deterministic worked demo, a failed producer is repaired; its changed output invalidates four
14 declared downstream nodes; two unaffected nodes retain prior evidence; and re-verification
15 accepts the repaired run. Fresh-store recovery reconstructs and validates the persisted hierarchy.
16 Under complete dependency declarations and version-bound evidence, a compact locality theorem
17 characterizes when unaffected work can be safely retained. The demonstrated results show how
18 evidence-gated verification can serve as a closed-loop control mechanism for bounded recovery in
19 long-horizon agent execution.

1 Verification as execution control

20
21 Agent loops already interleave reasoning, tool use, planning, and feedback [Yao et al., 2023, Shinn
22 et al., 2023, Zhou et al., 2024]. The verification problem becomes different when a run spans many
23 artifacts and contexts. A late “wrong” leaves unanswered: *what exactly was checked, which evidence*
24 *supported it, which later artifacts depended on it, and whether the next action is a retry, an upward*
25 *replan, or a human escalation?* Full restart is safe but wasteful; local retry is efficient but unsafe
26 if hidden dependencies exist. Reliability evaluation also shows that tool-using agents can fail in
27 ways that require environment-grounded checks rather than self-assessment alone [Debenedetti et al.,
28 2024].

29 We treat verification as an execution-time state transition, drawing on runtime-monitoring ideas
30 [Leucker and Schallhart, 2009]. Figure 1 summarizes the implemented object. The central contribu-
31 tion is the joint execution contract: persistent intent, bounded refinement, evidence lineage, explicit
32 verification outcomes, and an inspectable recovery scope that survives fresh-session recovery.

33 **Implemented control surface.** The supplied Python runtime uses package-local `STATE.json`
34 under `.ai-workflow/` as the authority index and JSON/Markdown records under `runtime/`.
35 Every Action is linked to its EPS, DIC, Phase, Plan, prompt node, result, and evidence paths. Evidence
36 paths are indexed back to the producing action. A recovery routine reconstructs and validates lineage
37 from package state without relying on the conversation that produced it.

2 Persistent contract, disposable execution

38
39 A **Plan** is the approved multi-phase authority surface. A **Phase** is a coherent milestone inside it.
40 Each Phase has exactly one **durable intent contract (DIC)** package: complementary views of the

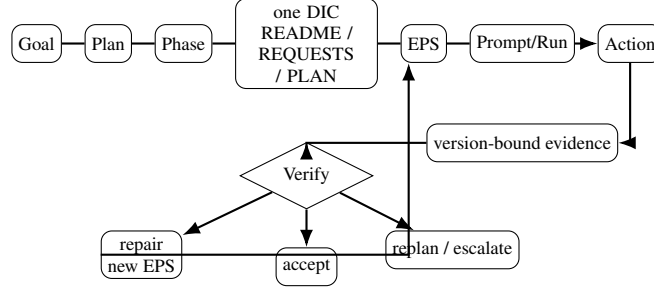


Figure 1: Implemented control surface. The DIC is the persistent multi-view Phase contract; an EPS is a disposable, state-specific realization. Evidence returns through a verification gate that changes execution rather than merely scoring a finished run.

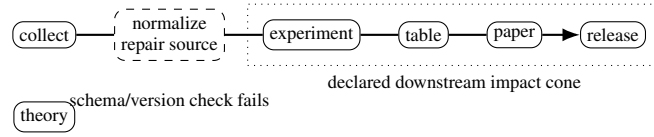


Figure 2: Deterministic worked demo. Repair changes the normalized key, so the second EPS contains the repair source and four downstream nodes; collect and independent theory evidence are retained. Evidence class: deterministic structural/runtime demonstration.

41 same persistent contract, with execution order encoded in `PLAN.md`. `README.md` summarizes
 42 the Phase; `REQUESTS.md` records concrete goals and acceptance targets; `PLAN.md` stores the
 43 durable prompt/dependency graph, including parallel subagents, dependencies, merge nodes, and
 44 conditional review/repair branches. `ARCHITECTURE.md` and `TEST_MATRIX.md` are materialized
 45 when relevant, and user-defined Markdown views are first-class.

46 A **disposable execution prompt stack (EPS)** is a just-in-time instantiation of DIC plan nodes
 47 for the current state and attempt. It records node dependencies, plan-node identity, executor,
 48 prompts/instructions, parallel groups, subagent flags, and stop conditions. Multiple EPS attempts can
 49 therefore refine the same DIC after new evidence arrives. This separation preserves durable intent
 50 while operational work is regenerated and re-verified.

51 **Four outcomes.** The runtime records `accept`, `repair`, `replan`, or `escalate`. `repair`
 52 keeps the Phase active so a new EPS can be generated. `replan` crosses the approved authority bound-
 53 ary through a protocol-change escalation, while `escalate` raises a material scientific/authority
 54 boundary. Thus a verifier participates in control flow even when it cannot itself repair the artifact.

55 3 Worked demo: failure to bounded recovery

56 We added a self-contained deterministic driver, `demo_trace.py`, that composes two mechanisms
 57 already present in the package: the core `WorkflowStore` hierarchy/evidence/verification runtime
 58 and the existing declared-dependency impact-cone helper.

59 The durable DIC contains seven plan nodes (Figure 2). The first EPS instantiates all seven and
 60 records versioned evidence. A deterministic verifier then rejects `normalize`'s output with a
 61 schema/version mismatch and records outcome `repair`. Repairing that producer changes the
 62 declared key `normalized`; `experiment` is its direct reader, so the declared downstream cone is
 63 `{experiment, table, paper, release}`. The second EPS instantiates the repair source plus
 64 those four nodes. `collect` and the independent `theory` node are not re-executed. Re-verification
 65 records `accept`, the Phase completes, and a fresh store reports package recovery as valid. The DIC
 66 identifier is unchanged across the two EPS attempts.

67 This trace establishes the demonstrated capability: a verification result changes subsequent execution,
 68 preserves inspectable lineage for retained work, and confines re-execution to an explicit declared
 69 dependency scope. Its evidence class is deterministic structural/runtime validation.

Table 1: Evidence currently present in the package. Categories are intentionally separated.

Evidence	Observed result	What it supports
Package-only recovery	<code>ok=true</code> ; clean	persistent-state recovery and lineage checks
Deterministic demo	<code>repair</code> $\rightarrow$ <code>accept</code> ; one DIC, two EPS attempts	structural closed-loop recovery under declared dependencies
Locality theorem	proved under explicit assumptions	conditional safety of retaining nodes outside the cone
Controlled agent benchmark	frozen protocol; execution pending	planned comparative evaluation

4 Why local retention can be sound

Let the durable execution graph be a DAG G . Node v declares a read set R_v and write set W_v . After repairing a source changes keys Δ , define the direct readers and downstream impact cone

$$B(\Delta) = \{v : R_v \cap \Delta \neq \emptyset\}, \quad \mathcal{I}(\Delta) = \text{Reach}_G(B(\Delta)).$$

Theorem 1 (Dependency-local recovery). *Assume complete declared dependencies/read-write footprints, evidence bound to input and artifact versions, deterministic execution under recorded parameters or fresh re-verification, and no undeclared mutable shared state. After a repair changes only Δ , completed outputs and evidence for every $v \notin \mathcal{I}(\Delta)$ remain contract-valid; re-execution need only be considered inside $\mathcal{I}(\Delta)$.*

Sketch. In topological order outside the cone, a node neither reads Δ nor depends on a changed predecessor; hence its declared inputs and predecessor outputs are unchanged. Its output/evidence therefore remains valid under the assumptions. The result adapts the dependency-locality principle used by incremental build systems [Mokhov et al., 2018]; one hidden edge invalidates the guarantee.

5 Validation, scope, and relation to prior work

Feedback agents such as Reflexion and search/planning agents such as LATS use external or self-generated feedback to improve later decisions [Shinn et al., 2023, Zhou et al., 2024]; automated design work searches over whole agent scaffolds [Hu et al., 2025]. Our systems contribution is the execution-time contract: what persists across attempts, how evidence is attached, and how a verification result chooses a bounded recovery or authority transition. Runtime-verification work motivates monitoring during execution [Leucker and Schallhart, 2009], while dependency-aware build systems motivate selective recomputation [Mokhov et al., 2018]. The distinctive systems object is their composition into a persistent, evidence-gated recovery contract.

Artifact surface and auditability. The runnable artifact exposes the state transitions directly. `demo_trace.py` builds a fresh temporary package, approves one Plan/Phase/DIC, records evidence-linked Actions for a seven-node EPS, records `repair`, computes the declared impact cone, executes the five-node repair EPS, records `accept`, completes the Phase, and reopens the package in a fresh store. `demo_trace.json` records the two EPS node sets, stable DIC, outcomes, retained nodes, and recovery result; the anonymous supplement contains the cleaned runtime, DIC/EPS schemas/templates, demo, and relevant verification/recovery tests.

Fresh-session recovery audit. Recovery validates Phase dependencies, DIC lineage and required views, the stored DIC hash, required report files, and every EPS–DIC binding. Malformed structures become errors and a legacy single-file DIC becomes a migration warning. Thus `recovery_ok=true` certifies structural consistency after reopening the persisted hierarchy; semantic validity remains delegated to verifier evidence.

Verification boundary. Evidence pointers establish lineage, while semantic validity remains the verifier’s responsibility. Verifier correctness, adversarial robustness of tool outputs, and dependency completeness are external assumptions. The demo explicitly composes the implemented impact-cone helper with `repair`; automatic impact-cone scheduling remains outside the core runtime.

107 **Planned empirical closure.** The package freezes an unexecuted comparative protocol spanning
108 direct/reactive execution (C0), flat plan-and-execute (C1), hierarchy without gated recovery (C2), and
109 hierarchy with verification-triggered repair/replanning (C3), with matched-budget controls. Tasks
110 vary dependency structure and injected faults; planned outcomes include unsupported completion,
111 detection latency, downstream propagation, recovery locality, and rework. These remain evaluation
112 commitments, so no comparative numbers appear here.

113 **Limitations.** Declared read/write dependencies can be incomplete, and the core runtime does
114 not infer them automatically. Verifiers can miss properties absent from their evidence surface;
115 independent is provenance rather than proof of statistical independence. Replanning crosses
116 a material authority boundary, the deterministic demo provides structural rather than confirmatory
117 comparative evidence, and the hierarchy may be unnecessary overhead for short tasks.

118 **Demo takeaway.** The implemented trace exposes a concrete systems pattern: evidence-bound
119 verification can turn failure into a localized, inspectable state transition with retained work and a
120 bounded recovery scope. Recovery itself therefore becomes an auditable object for long-horizon
121 agent development.

122 References

- 123 Edoardo DeBenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian
124 Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for
125 llm agents. In *Advances in Neural Information Processing Systems*, 2024.
- 126 Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International
127 Conference on Learning Representations*, 2025.
- 128 Martin Leucker and Christian Schallhart. A brief account of runtime verification. *The Journal of
129 Logic and Algebraic Programming*, 78(5):293–303, 2009. doi: 10.1016/j.jlap.2008.08.004.
- 130 Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. *Proceedings of
131 the ACM on Programming Languages*, 2(ICFP):79:1–79:29, 2018. doi: 10.1145/3236774.
- 132 Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion:
133 Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing
134 Systems*, 2023.
- 135 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.
136 ReAct: Synergizing reasoning and acting in language models. In *International Conference on
137 Learning Representations*, 2023.
- 138 Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language
139 agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the
140 41st International Conference on Machine Learning*, 2024.